

if-else-satser

Ibland kan det vara praktiskt att kunna utföra en del av ett program om en `if`-sats evaluerats till `false`, d.v.s. om den har inte utförts. Det är förstås möjligt att skriva en andra `if`-sats som kontrollerar det neget uttrycket jämfört med den första, men det hela kan göras lättare genom att skriva en `else`-del till `if`-satsen i fråga. Denna `else`-del utförs endast om `if`-satsen evaluerades till `false`. Den allmänna formen är:

```
if ( villkor )
    sats1;
else
    sats2;
```

I en `if`-sats garanteras att antingen `if`-delen *eller* `else`-delen utförs, aldrig båda eller ingendera. Ett exempel med talet 27 som fanns tidigare kunde bli:

```
// ge en variabel Tal ett värde
int Tal;
cin >> Tal;
if ( Tal == 27 ) {
    // talet var 27
    cout << "Talet var 27!\n";
}
else {
    // talet var inte 27
    cout << "Talet var inte 27!\n";
}
```

Det går även att kombinera ihop flera `if`-satser med varandra för att åstadkomma en lång rad olika fall. Det hela belyses bäst med ett exempel:

```
// ge en variabel Tal ett värde
int Tall;
cin >> Tal;
if ( Tal < 20 ) {
    // talet är mindre än 20
    cout << "Talet är mindre än 20!\n";
}
else if ( Tal < 25 ) {
    // talet är mindre än 25, man större eller lika med 20
    cout << "Talet är större eller lika med 20, men mindre än 25!\n";
}
else {
    // talet är större eller lika med 25
    cout << "Talet är större eller lika med 25!\n";
}
```

Exemplet ovan är kanske inte så speciellt realistiskt, men illustrerar hur en ny `if`-sats inkluderas i `else`-delen av en annan sats. Den sista `else`-delen utförs endast om inget av de ovanstående villkoren uppfyllts.

[Förutgående](#)
Flödeskontroll

[Hem](#)
[Upp](#)

[Nästa](#)
Logiska operatorer